

KAPITEL 3: HTML

LERNZIELE

- Die prinzipielle Syntax und den Aufbau von HTML beschreiben können.
- Erläutern können, wie HTML und CSS zusammen arbeiten.
- Wichtige Head-Informationen kennen und diese in HTML spezifizieren können.
- Texte mit HTML strukturieren und formatieren können.
- Listen, Hyperlinks und Tabellen in HTML kennen und anwenden können.
- Formulare und Formularelemente kennen und anwenden können.
- Block- und inline-Elemente kennen sowie den Unterschied zwischen beiden erklären und anwenden können.



3.1 ÜBERBLICK

Vorbemerkungen:

- Die aktuelle Version HTML5 besteht aus HTML, CSS3 und JavaScript.
- Die Demo HTML5 rocks gibt einen Überblick über die Features.
- Aufgrund der Feature-Vielzahl besprechen wir hier nur einen Ausschnitt!

HTML ...

- ist eine Sprache zur Beschreibung von Hypermedia-Dokumenten, die typischerweise im WWW verwendet wird.
- bedeutet Hypertext Markup Language
- basierte bis einschließlich Version 4 auf SGML (Standardized General Markup Language), eine Metasprache zur Beschreibung von Markup-Sprachen wie HTML und XML.

Ein **Markup** (engl. für *Auszeichnung/Hervorhebung*) – auch **Tag** genannt – ist ein Steuerelement zur Formatierung und Plazierung von Inhalten (Text, Bild, Video, Audio).

Eigenschaften von Markup-Sprachen wie HTML bzw. XML sind:

- es gibt öffnende und schließende Tags, z.B. body

```
<body> ... </body>
```

- innerhalb von Tags können weitere Tags stehen oder einfacher Text:

```
<body>
  <h1>Hallo!</h1>
  Hier ist meine Geschichte ...
</body>
```

- Tags können Attribute haben, z.B. "href"

```
<a href="http://www.thi.de">zur THI</a>
```



Die HTML5-Syntax ...

HTML5 basiert aufgrund seiner komplexen Syntax nicht mehr auf der Standardized General Markup Language (SGML), da HTML5 nicht mehr mit SGML beschrieben werden kann.

Die HTML5-Syntax ist z.B. hier beschrieben und wird durch gängige Browser in Form eines HTML-Interpreters implementiert. Es werden aber je nach Browser nicht alle Elemente unterstützt!

Zur Geschichte von HTML:

- 1992 von Tim Berners-Lee am CERN festgelegt
- 1997 HTML 4.0 mit Stylesheets, Skripten (JavaScript) und Frames; Frames werden heute nicht mehr unterstützt – Beispiele siehe etwa SelfHTML
- 2000 XHTML (extensible HTML)
    - auf XML (statt SGML) -basierende Variante von HTML 4
    - Vorteil: einfacher zu parsen, da jedes Tag auch wieder geschlossen wird (in HTML nicht immer nötig!)
- seit 2008: HTML5

Das folgende kleine Beispiel soll einen ersten Eindruck von HTML geben:

```
<!DOCTYPE html>
<html>
<head>
    <title>Hello, World!</title>
</head>
<body>
    <h1>This is a header 1</h1>
    <hr />
    <p>This is some Text!</p>
    <p>
        <a href='http://www.thi.de' target='_blank'>
            This is a link
        </a>
    </p>
</body>
</html>
```

- Nachfolgend wird ein HTML5-Dokument definiert.
- Das HTML-Dokument mit Kopf- (head) und Inhaltsbereich (body)
- Kopfbereich für Titel, Style-Sheets und Metainformationen
- Definiert eine Überschrift der Größe 1.
- Definiert mehrere Absätze (p) mit einem Hyperlink (a)



Wie lernt man HTML?

Neben vielen anderen ist das W3C-Tutorial zu empfehlen:

- Zu vielen Abschnitten gibt es Beispiele zum Ausprobieren sowie kurze Tests zum Überprüfen des Gelernten.
- Am Ende gibt es ein HTML-Quizz, wo man das Gelernte ebenfalls noch einmal überprüfen und vertiefen kann.



Tipps:

- Beim Surfen um Web können Sie ebenfalls HTML lernen! Einfach mit der rechten Maustaste zu der gerade angezeigten Seite den Quelltext anzeigen lassen…
- Professioneller ist es, in Chrome oder Firefox die Entwicklertools anzeigen zu lassen, damit können Sie die aktuelle Seite nach vielen Kriterien analysieren. Dazu auf einem HTML-Element > rechte Maustaste > Element untersuchen klicken.

online-Informationen zu HTML5

- Tag-Referenz: http://www.w3schools.com/tags/
- SELF HTML
  - Tutorials: https://wiki.selfhtml.org/wiki/HTML/Tutorials
  - Tag-Übersicht: http://www.selfhtml5.org/html5-tag-systematik/
- Bücher: gibt es viele – bitte bei Bedarf selbst eines aussuchen!

3.2 DOKUMENTAUFBAU UND SYNTAX

HTML-GRUNDIDEE

Mit HTML wird folgender **Modularisierungsansatz** verfolgt:

- Jedes HTML-Dokument trennt den Inhalt von der Darstellung des Inhalts, also dem Layout bzw. Styling.
- Inhalt wird mit der Sprache HTML formuliert, während die Darstellung mit CSS (cascading style sheets, siehe nächstes Kapitel) spezifiziert wird.

Beispiel - gegeben sei folgende HTML-Seite:

```
<!DOCTYPE html>
<html>
<head>
    <link rel='styleheet' type='text/css' href='style.css'>
    <meta charset='UTF-8'>
</head>
<body>
    <h1>Überschrift mit Größe 1</h1>
    <hr>
    <p>
        Wird das Stylesheet eingebunden, so formatiert es den Text.
        <a href='http://www.thi.de'>Link zur THI-Homepage</a>
    </p>
</body>
</html>
```

Ohne das Stylesheet `style.css` wird die Webseite mit Serifer Schrift ohne weitere Veränderung angezeigt.

Nun sei folgendes Stylesheet gegeben, die genaue Erklärung der Regelsyntax erfolgt im nächsten Kapitel. Die Kommentare geben einen Eindruck, was gestyle't werden soll:

```
body {  /* Styles für das body-Tag */
    background-color: lightgray; /* Seite mit hellgrauem Hintergrund */
}
h1 {    /* Styles für das h1-Tag */
    color: red;          /* Überschrift mit roter Schrift */
    font-size: 20 px;    /* ...und 20 Pixel groß */
}
hr { /* Styles für das hr-Tag */
    color: white; /* weisse Trennlinie */
}
a:visited {     /* Styles für das a-Tag im Zustand visited */
    color: gray; /* geklickter Hyperlink mit grauer Schrift */
}
```

Mit dem Ergänzen von Styles wird der Seite das gewünschte Aussehen verliehen.

HTML-DOKUMENTAUFBAU

```
<!DOCTYPE html>
<html>
    <head>
        <!--
            ...
        -->
    </head>
    <body>
        <!--
            ...
        -->
    </body>
</html>
```

- Für HTML5-Dokumente sollte in der ersten Zeile der Dokumenttyp genannt werden. Dies erleichtert die Interpretation des Dokuments im Browser.
- Anschließend wird Kopfinformation (head),
- und der eigentlich anzuzeigende Inhalt (body) bereitgestellt.

Beispiele im body-Tag zeigen die Tag-Syntax:

```
...
<body>
    <tag attr1="wert1" attr2="wert2">
        <!-- Kommentar - ggf. über mehrere Zeilen -->
        <tagOhneSchliessendesTag attr3="wert3">
        <tagMitKurzemAbschluss />
        Mathe: 3 &lt; 5!
    </tag>
</body>
...
```

- Tags stehen in spitzen Klammern und können Attribute haben.
- Typischerweise wird ein Tag geöffnet und wieder geschlossen.
- Es gibt aber auch Ausnahmen (z.B. input).
- Das schließende Tag kann verkürzt dargestellt werden, wenn zwischen den beiden Tags nichts stehen soll.
- Zu beachten sind die HTML-Sonderzeichen - hier wird das Kleiner-Zeichen dargestellt.

HTML ENTITIES

In HTML müssen bestimmte Zeichen wie `<`, `>`, `&`, `"` und `'` maskiert werden, da sie Teil der HTML-Syntax sind und sonst vom Browser als Steuerzeichen interpretiert werden. Um diese Zeichen im Fließtext korrekt darzustellen, verwendet man sogenannte HTML-Entities:

- `<` für das Kleiner-Zeichen <
- `>` für das Größer-Zeichen >
- `&` für das kaufmännische Und &
- `"` für das doppelte Anführungszeichen "
- `'` für das einfache Anführungszeichen '

Weitere Informationen und eine vollständige Liste der HTML-Sonderzeichen finden Sie auf w3schools.

HEAD-INFORMATIONEN

```
<head>
    <title>Mein erstes HTML-Dokument!</title>
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="description" content="Dies ist eine Beispielbeschreibung für
</head>
```

- Titel für die Benennung des Tabs im Browserfenster
- Viewport als Meta-Information für die Steuerung der Darstellung auf mobilen Geräten
- Meta-Information: Optionale Angaben für Browser, Server u. Suchprogramme, z.B. eine Beschreibung:

Beispiele für Meta-information im Header

- Zeichensatz für deutsche Umlaute einstellen:

```
<meta charset="UTF-8">
```

- Gültigkeit bis (steuert das Caching im Browser):

```
<meta http-equiv="expires" content="01 Dec 2006 00:00:00 GMT">
```

- Schlüsselworte für Suche:

```
<meta name="keywords" content="Kleidung, Schuhe, Mode">
```

Beispiele für Meta-information im Header (Fortsetzung)

- Logische Beziehungen zu anderen Dateien: Einbindung Stylesheet(s) und JavaScript-Skript(e):

```
<link rel="stylesheet" type="text/css" href="../../src/style.css">
<script src="javascript-test.js"></script>
```

- Basis-Adresse festlegen für relative Adressierung:

```
<base href="http://localhost">
```

- Links in neuem Fenster anzeigen (sonst Laden in aktuellem Browsertab):

```
<base target="_blank">
```

ÜBUNG: HTML

Welche Aussagen stimmen?

- [ ] HTML bedeutet (H)TTP (T)ransferable (M)arkup (L)anguage
- [ ] Der Inhalt-Teil eines HTML-Dokuments strukturiert verschiedene CSS-Anweisungen
- [ ] HTML bedeutet (H)yper(t)ext (M)arkup (L)anguage
- [ ] Ein vollständiges HTML-Dokument enthält Dokumenttyp, Kopfinformationen und Inhalt

ÜBUNG: HTML VS. CSS

Welche Vorteile ergeben sich durch die Aufteilung in HTML und CSS?

- [ ] Der Entwickler kann sich entscheiden, wo er welche Information plazieren möchte.
- [ ] HTML beschreibt nur das Layout, während man mit CSS getrennt davon den Inhalt beschreibt. Dadurch wird der Code besser wartbar.
- [ ] Die Trennung von Inhalt und Styling bringt alle Vorteile eines modularen Ansatzes mit sich.

ÜBUNG: GRUNDAUFBAU EINER HTML-SEITE

Erklären Sie anhand eines einfachen Beispiels, was die Grundstruktur für ein HTML-Dokument ist und wie dieses erstellt bzw. gespeichert wird.

Geben Sie eine eigene Antwort, eine Beispielantwort wird nach dem Absenden angezeigt

3.3 TEXTSTRUKTURIERUNG UND -FORMATIERUNG



Zur Anzeige von Text gibt es eine Reihe von HTML-Tags, die im Folgenden vorgestellt werden. Alle übrigen Text-Attribute wie beispielsweise Zeichensätze werden mit CSS gesetzt.

ÜBERSCHRIFTEN

```
<h1>Überschrift Größe 1</h1>
<h2 align='right'>Überschrift Größe 2 rechtsbündig</h2>
<h3 style='color:blue;'>Überschrift Größe 3 in blau</h3>
<h4>Überschrift Größe 4</h4>
<h5>Überschrift Größe 5</h5>
<h6>Überschrift Größe 6</h6>
```

- Es gibt 6 vordefinierte Größen für Überschriften.
- Über das Attribut `align` kann eine Überschrift ausgerichtet werden (Werte: left, right, center). Nutzbar, aber in der Praxis nicht mehr relevant.
- Sog. "inline styling" (siehe Kapitel CSS) kann genutzt werden, um beispielsweise die Textfarbe auf blau zu setzen.

ABSÄTZE

```
Text vor p
<p>
    Erste Zeile <br> Zweite Zeile
</p>
Text nach p
```

- Absätze werden mit dem `p`-Tag (engl. paragraph) definiert.
- Manueller Umbruch: der Browser bricht standardmäßig automatisch am rechten Rand um, mit `br` kann man dies beeinflussen.

  Hinweis: Die blaue Markierung dient hier nur der visuellen Unterstützung.

RECHTECKE MIT/OHNE ZEILENVORSCHUB

```
Text vor div
<div>
    Ein Block mit Text in einem div,
    hier <span>gestylte
    Worte mit span</span>
</div>
Text nach div
```

- Allgemeiner *block* Container: `div` definiert einen Container ohne semantische Bedeutung, welcher beliebig befüllt und gestyle't werden kann.
- Allgemeiner *inline* Contanier: `span` definiert ebenfalls einen Container ohne Vorschub davor und danach.

Hinweis: Roter Rahmen und die blaue Schriftfarbe dient hier nur der Übersicht.

SCHRIFTDIREKTIVEN

```
<i>kursive Schrift</i><br>
<b>Fettdruck</b><br>
<u>Fettdruck</u>
<p style='font-family:monospace'>
    Schrift mit fester Breite, in HTML4 mit tt-Tag</p>
```

- Kursive Schrift mit `i`-Tag
- Fettgedruckte Schrift mit `b`-Tag
- Unterstrichene Schrift mit `u`-Tag
- Zeichensatz mit fester Breite, hier mit inline-style
- Diese HTML-Tags nur formatierende Auswirkung.



Alternative semantische Tags

```
<em>kursive Schrift</em><br>
<strong>Fettdruck</strong><br>
<mark>Markiert</mark>
```

- Kursive Schrift mit `em`-Tag, betont etwas
- Fettgedruckte Schrift mit `strong`-Tag, markiert größere Bedeutung
- Fettgedruckte Schrift mit `mark`-Tag, markiert Begriffe die z.B. im Rahmen einer Suche relevant sind

Worin liegt der Unterschied zwischen `b`, `i`, `em` und `strong`?

- Die Tags `b`, `i` und `u` drücken explizit aus, dass Text fett, kursiv oder unterstrichen sein soll
- Die Tags `em` und `strong` haben eine semantische Bedeutung (zur Betonung, Stärkung der Bedeutung)
- Visuell unterscheiden sich `b` und `strong`, sowie `i` und `em` nicht
- Mit Blick auf CSS sollte der Einsatz von HTML Tags, die *nur* dem Styling dienen, mit bedacht passieren

Weitere typische Tags

```
<small>Kleine Schrift</small>
<big>Große Schrift, in HTML4 mit big-Tag</big>
Dann noch hochgestellt x<sup>2</sup>
bzw. tiefgestellt a<sub>i, j</sub>
```

- kleinere Schrift
- größere Schrift
- Hochgestellte / tiefgestellte Schrift durch `sup`- bzw. `sub`-Tag

VERSCHIEDENES

```
Hier der Quellcode:
<pre>
    let array = [];
    if (x > 1) {
        array.append(x);
    }
</pre>
```

Quellcode kann mit `pre` unverändert (d.h. unter Erhaltung der Leerzeichen etc.) dargestellt werden.

```
Text vor der Trennung
<hr>
Dieser Text ist <mark>hervorgehoben!</mark>
```

- Das Tag `hr` (horizontal ruler) setzt eine Trennlinie.
- Markierungen gelingen mit `mark`.

ÜBUNG: HTML-VERSTÄNDNIS (1)

Welches HTML-Element wird verwendet, um eine Überschrift der höchsten Ebene zu erstellen?

- [ ] <h1>
- [ ] <h6>
- [ ] <p>
- [ ]

ÜBUNG: HTML-VERSTÄNDNIS (2)

Welches Element wird verwendet, um einen inline-Bereich (ein Container ohne Vorschub) zu definieren der keinen zusätzlichen Stil hat?

- [ ]
- [ ] <section>
- [ ]
- [ ] <p>

ÜBUNG: HTML-VERSTÄNDNIS (3)

Welches Element dient dazu, einen Teil des Textes als wichtig (betont / besonders) hervorzuheben, ohne ihn notwendigerweise fett darzustellen?

- [ ] <i>
- [ ]
- [ ]
- [ ] <b>

ÜBUNG: HTML-VERSTÄNDNIS (4)

Beschreiben Sie den Unterschied zwischen semantischem Markup und formatierendem Markup in HTML. Geben Sie jeweils zwei Beispiele und erklären Sie, warum die Verwendung von semantischem Markup empfohlen wird.

Geben Sie eine eigene Antwort, eine Beispielantwort wird nach dem Absenden angezeigt

3.4 LISTEN



HTML unterstützt die Definition von Aufzählungslisten, nummerierten Listen und Definitionslisten.

AUFZÄHLUNGSLISTEN

```
<h3>Mein Obstkorb:</h3>
<ul style='list-style-type: circle'>
    <li>3 Äpfel</li>
    <ul>
        <li>1 roter Apfel</li>
        <li>2 grüne Äpfel</li>
    </ul>
    <li>4 Bananen</li>
</ul>
```

- Listentag `ul` (unordered list), `list-style-type` definiert das Aufzählungszeichen
- Aufzählungselement mit Tag `li` (engl. list item)
- Listen können geschachtelt werden.

Wichtige Typen für das list-style-type-Attribut

- `disc` – gefüllter Kreis (Standard)
- `circle` – leerer Kreis
- `square` – Quadrat
- `none` – kein Aufzählungszeichen

Weitere Werte und Beispiele findet man bei W3Schools

NUMMERIERTE LISTEN

```
<h4>Nummerierte Liste, römisch</h4>
<ol type='I'>
    <li>bei Bianca vorbeischauen</li>
    <li>bei Christine vorbeischauen</li>
</ol>
<h4>Nummerierte Liste mit Startwert 7</h4>
<ol start='7'>
    <li>bei Bianca vorbeischauen</li>
    <li>bei Christine vorbeischauen</li>
</ol>
```

- Listentag `ol` (engl. ordered list)
- Attribut `type` zur Definition der Nummerierungsart
- Attribut `start` zur Festlegung eines Startwerts

Wichtige Werte für das type-Attribut von

- `1` – Zahlen (Standard)
- `a` – kleine Buchstaben (a, b, c, ...)
- `A` – große Buchstaben (A, B, C, ...)
- `i` – kleine römische Zahlen (i, ii, iii, ...)
- `I` – große römische Zahlen (I, II, III, ...)

Weitere Werte und Beispiele findet man z.B. bei W3Schools

DEFINITIONSLISTEN

```
<h4>Definitionsliste</h4>
<dl>
    <dt>AA</dt>
    <dd>Auto Answer (Modem)</dd>
    <dt>AAE</dt>
    <dd>Allgemeine Anschalte-Erlaubnis</dd>
    <dt>AARP</dt>
    <dd>Appletalk Address Resolution Protocol</dd>
</dl>
```

- Listentag `dl` (engl. definition list)
- Abkürzung mit `dt` (engl. definition title)
- Erklärung mit `dd` (engl. definition description)

ÜBUNG: LISTEN-TAGS

Welche der folgenden Gruppen enthält NUR Elemente, die direkt mit HTML-Listen zusammenhängen?

- [ ] , <li>, <ol>, <dt>, <dd>
- [ ] , <li>, <ol>, <b>, <i>
- [ ] <dl>, <dt>, <dd>, ,
- [ ] , <li>, <ol>, <a>, <p>

ÜBUNG: VERSCHACHTELTE LISTE

In einer verschachtelten Liste, wo sollte das <ul> oder <ol> Tag für die untergeordnete Liste platziert werden?

- [ ] Außerhalb der gesamten übergeordneten Liste
- [ ] Innerhalb des übergeordneten <li> Tags
- [ ] Vor dem übergeordneten <li> Tag
- [ ] Nach dem übergeordneten <li> Tag



3.5 HYPERLINKS



Hyperlinks geben den Hypertexten ihren Namen und sind i.d.R. Verweise auf andere Hypertext-Dokumente.

Einen Hyperlink definiert man mit Hilfe des `a`-Tags, a steht für engl. *anchor*:

```
<a href="<URL>">Verweistext zum Anklicken</a>
```

Mit dem HTML-Attribut `href` (Hypertext Reference) wird dann der Verweis angegeben.

GRUNDSÄTZLICHE NUTZUNG VON HYPERLINKS

Das href-Attribut ist das wichtigste Attribut für Hyperlinks, es gibt die Zieladresse des Hyperlinks an. *Beispiel: Die Navigation zur THI-Homepage*

```
Zur Technischen Hochschule Ingolstadt klicke <br>
<a href='http://www.thi.de'>hier (aktuelles Tab)</a> oder <br>
<a href='http://www.thi.de' target='_blank'>hier (neues Tab)</a>
```

- Klickt man auf das Wort "hier (aktuelles Tab)", so wird die Zielseite im aktuellen Browser-Tab geladen.
- Durch die Angabe `target="_blank"` öffnet sich die Zielseite in einem neuen Browser-Tab.



Absolute Adressierung

```
<a href="http://www.thi.de/irgend/seite.html">Link</a>
```

Ein Link-Ziel mit Schema, Domain und optional einem Pfad als **absoluter** Link bezeichnet. Die angegebene Ressource wird mit dem Link aufgerufen.

Absolute Adressierung ohne Domain

```
<a href="/ein/pfad/zur/datei.html">Link</a>
```

Ein Link-Ziel ohne Domain aber mit führenden `/` wird ist ebenso ein **absoluter** Link. Der Browser orientiert sich am aktuellen Dokument und leitet daraus das Link-Ziel ab. Dabei wird in der aktuellen URL der komplette Pfad ersetzt.

Beispiel: Wird aktuell die Seite `http://www.thi.de/irgend/seite.html` im Browser angezeigt, ergibt sich aus dem Link anschließend `http://www.thi.de/ein/pfad/zur/datei.html`.



Relative Adressierung

```
<a href="ein/pfad/zur/datei.html">Link</a>
```

Ein Link-Ziel ohne Domain und ohne führenden `/` wird als **relativer** Link bezeichnet. Der Browser orientiert sich am aktuellen Dokument und leitet daraus das Link-Ziel ab. Dabei wird der Pfad der aktuellen URL mit dem Pfad des Links kombiniert.

Beispiel: Wird aktuell die Seite `http://www.thi.de/irgend/seite.html` im Browser angezeigt, ergibt sich aus dem Link anschließend `http://www.thi.de/irgend/ein/pfad/zur/datei.html`.

HYPERLINKS MIT GRAFIKEN

Häufig werden Grafiken zum Anklicken verwendet, dies kann durch ein eingebettetes img-Tag erreicht werden.

```
Klicke das Logo und öffne THI in einer neuen Seite:
<a href='http://www.thi.de' target='_blank'>
    <img src='images/thi_logo_wb_RGB.svg'
        alt='logo'
        width='186' height='83'>
</a>
```

- Mit dem `img`-Tag wird eine Grafik eingebunden.
- Ist das Bild nicht ladbar, wird der durch `alt` definierte Text angezeigt.
- Die Attribute `width` und `height` legen die Pixel-Größe der Grafik fest.

BUTTONS ALS HYPERLINKS

Buttons werden zum Anstossen von Aktionen eingesetzt, typischerweise, um ein Formular mit Daten zum Server zu senden.

Buttons können aber auch wie ein Hyperlink zur Navigation genutzt werden. Hierfür muss der Button in ein "a"-Tag eingebettet werden.

```
Drücke den Button, um zu THI-Homepage zu gelangen:
<a href='http://www.thi.de' target='_blank'>
    <button>zur THI!</button>
</a>
```

- Das `button`-Tag definiert einen Button mit dem Text "zur THI!".
- Durch die Einbettung arbeitet der Button wie ein Link.

ÜBUNG: RELATIVE VERLINKUNG

Wie sieht eine korrekte relative Verlinkung zu einer Datei namens "kontakt.html" aus, die sich im übergeordneten Verzeichnis befindet?

- [ ] "./kontakt.html"
- [ ] "kontakt.html"
- [ ] "/kontakt.html"
- [ ] "../kontakt.html"

ÜBUNG: ABSOLUTE UND RELATIVE VERLINKUNG

Welche der folgenden Aussagen über relative und absolute Links sind FALSCH?

- [ ] Relative Links sind weniger wartungsfreundlich als absolute Links
- [ ] Relative Links funktionieren nur innerhalb derselben Website
- [ ] Absolute Links beginnen immer mit einem Protokoll (z.B. http:// oder https://)
- [ ] Absolute Links beginnen immer mit einem Schrägstrich "/", ohne ein Protokoll oder eine Domain anzugeben



3.6 BILDER

Bilder sind ein wesentlicher Bestandteil moderner Webseiten und können mit dem `img`-Tag in HTML-Dokumente eingebunden werden.

Ein Bild definiert man mit Hilfe des `img`-Tags:

```
<img src="<Pfad-zum-Bild>" alt="Beschreibung des Bildes">
```

Mit dem HTML-Attribut `src` (Source) wird der Pfad zur Bilddatei angegeben, während das `alt`-Attribut eine alternative Textbeschreibung für das Bild bereitstellt.

GRUNDSÄTZLICHE NUTZUNG VON BILDERN

Das src-Attribut ist das wichtigste Attribut für Bilder, es gibt den Pfad zur Bilddatei an.

Beispiel: Einbindung eines Fotos

```
<img src='images/photo1.jpg' alt='Standfoto' width='256' height='256'>
```

Das erste Bild wird von einem relativen Pfad geladen (relativ zum HTML-Dokument).

Absolute Adressierung

```
<img src="https://www.example.com/images/logo.png" alt="Beschreibung">
```

Ein Bild-Pfad mit Schema, Domain und optional einem Pfad wird als **absoluter** Link bezeichnet. Das Bild wird direkt von der angegebenen URL geladen.

Absolute Adressierung ohne Domain

```
<img src="/images/logo.png" alt="Beschreibung">
```

Ein Bild-Pfad ohne Domain aber mit führenden `/` ist ebenso ein **absoluter** Link. Der Browser orientiert sich an der aktuellen Domain.

Beispiel: Wird aktuell die Seite `http://www.thi.de/irgend/seite.html` im Browser angezeigt, ergibt sich aus dem Bild-Pfad anschließend `http://www.thi.de/images/logo.png`.

Relative Adressierung

```
<img src="images/logo.png" alt="Beschreibung">
<img src="../images/logo.png" alt="Beschreibung">
```

Ein Bild-Pfad ohne Domain und ohne führenden `/` wird als **relativer** Link bezeichnet. Der Browser orientiert sich am aktuellen Dokument und leitet daraus den Bild-Pfad ab.

Beispiel: Wird aktuell die Seite `http://www.thi.de/irgend/seite.html` im Browser angezeigt, ergibt sich aus dem ersten Bild-Pfad anschließend `http://www.thi.de/irgend/images/logo.png` und aus dem zweiten Bild-Pfad anschließend `http://www.thi.de/images/logo.png`.

WICHTIGE ATTRIBUTE

alt-Attribut (Alternative Text)

- Wird angezeigt, wenn das Bild nicht geladen werden kann
- Wichtig für Barrierefreiheit (Screenreader)
- Sollte das Bild beschreiben oder seinen Zweck erklären

width und height

- Legen die Anzeigegröße des Bildes in Pixeln fest
- Helfen beim Layout und vermeiden "Springen" der Seite beim Laden
- Kann später mittels CSS gesetzt werden

title-Attribut

- Zeigt einen Tooltip beim Hovern über das Bild
- Optional, aber nützlich für zusätzliche Informationen

```
<img src='images/photo2.jpg'
     alt='Unser neuestes Smartphone in silberner Farbe'
     title='Ich bin ein Foto'
     width='256'
     height='256'>
```

- Das `alt`-Attribut beschreibt das Bild aussagekräftig.
- Das `title`-Attribut zeigt einen Tooltip
- Die Attribute `width` und `height` definieren die Anzeigegröße.

UNTERSTÜTZTE BILDFORMATE

Moderne Browser unterstützen verschiedene Bildformate:

- **JPEG/JPG**: Ideal für Fotos mit vielen Farben
- **PNG**: Unterstützt Transparenz, gut für Logos und Grafiken
- **GIF**: Unterstützt Animationen, begrenzte Farbpalette
- **SVG**: Vektorgrafiken, skalierbar ohne Qualitätsverlust
- **WebP**: Modernes Format mit besserer Kompression

ÜBUNG: ALT-ATTRIBUT BEI BILDERN

Warum ist das alt-Attribut bei Bildern wichtig?

- [ ] Es verbessert die Barrierefreiheit für Screenreader
- [ ] Es ist optional und hat keine praktische Bedeutung
- [ ] Es macht das Bild schneller ladbar
- [ ] Es ist nur für die Suchmaschinenoptimierung wichtig

ÜBUNG: BILDPFADE

Welche der folgenden Aussagen über Bildpfade sind korrekt?

- [ ] Das width-Attribut kann helfen, das Layout während des Ladens zu stabilisieren
- [ ] Das alt-Attribut ist nur bei lokalen Bildern erforderlich
- [ ] Relative Pfade sind besser für die Portabilität einer Website
- [ ] Absolute Pfade mit Domain funktionieren auch bei externen Bildern



3.7 TABELLEN

ELEMENTARE TABELLENDARSTELLUNG

Tabellen dienen der Darstellung von Daten in Matrixform. Beispiel:

```
<table style='border: 2px dotted black;'>
    <tr><th>Name</th><th>Alter</th><th>Ort</th></tr>
    <tr><td>Peter Maier</td><td>25</td><td>85049 Ingolstadt</td></tr>
    <tr><td>Sylvie Schmied</td><td>31</td><td>83667 München</td></tr>
</table>
```

- Eine Tabelle wird mit dem `table`-Tag definiert.
- Der inline-style setzt einen 2 Pixel dicken schwarz gepunkteten Rand.
- Tabellenzeilen werden mit dem `tr`-Tag (table row) definiert.
- Tabellenüberschriften definiert man mit dem `th`-Tag (table heading).
- Eine Zelle mit Inhalt wird mit dem `td`-Tag (table data) definiert.

Wichtige Anmerkung

Das Grundlayout einer Webseite kann mit Hilfe von geschachtelten Tabellen gestaltet werden.

Die `div`-Tags sind jedoch besser geeignet, da sie mehr Flexibilität gerade im Hinblick auf unterschiedliche Darstellungsmedien (z.B. Smartphone, Print) bieten und mit CSS nach belieben gestylt werden können.

KOMPLEXERE TABELLENDARSTELLUNG

```
<table>
    <caption>Personaltabelle</caption>
    <thead>
        <tr><th>Name</th><th>Alter</th></tr>
    </thead>
    <tbody>
        <tr><td>Petra Maier</td><td>21</td></tr>
        <tr><td>Eva Müller</td><td>19</td></tr>
    </tbody>
    <tfoot>
        <tr><td colspan='2'>erstellt am 19.02.2015</td></tr>
    </tfoot>
</table>
```

- Die getrennte Definition von Kopf, Körper und Fußzeile ermöglicht ein flexibleres Layout (z.B. Wiederholung der Überschriften auf Folgeseiten).
- Durch `colspan` erstreckt sich die letzte Zelle über 2 Spalten. Analog gibt es `rowspan` für Zeilen.

Tabellenstyling

Durch Vorab-Definition der Spalten mit "colgroup" bzw. "col" können Spalten gemeinsam gestyle't werden, siehe W3Schools.

Das Styling mit CSS ist für Tabellen sehr wichtig, dies wird im CSS-Kapitel gezeigt.

ÜBUNG: TABELLEN-TAGS

Welche der folgenden Gruppen enthält NUR Elemente, die üblicherweise zur Struktur einer HTML-Tabelle gehören?

- ☐ <table>, <tr>, <td>, <th>, <thead>
- ☐ <table>, <tr>, <td>, ,
- ☐ <table>, <thead>, <tbody>, <col>, <p>
- ☐ <table>, <row>, <cell>, <header>, <body>

ÜBUNG: TABELLEN-BEREICHE

Welche der folgenden Aussagen über <tbody>, <thead> und <tfoot> in HTML-Tabellen ist **korrekt**?

- [ ] <tbody> kann mehrfach in einer Tabelle verwendet werden, <thead> und <tfoot> jedoch nur einmal.
- [ ] <thead> muss immer vor <tbody> und <tfoot> stehen, sonst wird die Tabelle nicht korrekt gerendert.
- [ ] Die Verwendung von <tbody>, <thead> und <tfoot> ist verpflichtend für jede HTML-Tabelle.
- [ ] <tfoot> wird immer am Ende der Tabelle angezeigt, unabhängig von seiner Position im HTML-Code.



3.8 FORMULARE



Formulare dienen der Dateneingabe und der anschließenden Übermittlung (engl. submit) an den Webserver, der aus den Eingabedaten typischweise eine Antwortseite generiert und zum Browser zurückschickt.

Syntax und Funktionsweise werden anhand eines Beispielformulars gezeigt.

```
<form action='verarbeiteDaten.php' method='GET'>
    <label for='vornameFeld'>Vorname:</label><br>
    <input type='text' id='vornameFeld'
        name='vorname' value='Codie' required><br>
    <label for='nachnameFeld'>Nachname:</label><br>
    <input type='text' id='nachnameFeld'
        name='nachname' value='Coder'><br><br>
    <input type='submit' value='Senden'>
</form>
```

Jedes Formular startet mit einem `form`-Tag, dieser umfasst alle Eingabemöglichkeiten und definiert was mit den Eingaben passieren soll.

Hier sollen nach dem Klick auf den Senden-Button die Daten zu einem Webserver gesendet werden und dort von einem PHP-Skript namens `verarbeiteDaten.php` verarbeitet werden.

Würde man das Beispiel lokal ausprobieren und den Senden-Button verwenden, erscheint folgende Seite:



Das Problem ist, dass es die Datei "verarbeiteDaten.php" nicht gibt und wir keinen geeigneten Webserver mit PHP nutzen.

Interessant ist an obiger Fehlermeldung, dass das PHP-Skript "verarbeiteDaten.php" mit einem Query-String als Parameter aufgerufen wird. Der Query-String ist der Text hinter dem Skript-Namen, also

```
?vorname=Codie&nachname=coder
```

Die eingegebenen Daten wurden also als Schlüssel/Wert-Paare übergeben. Die Festlegung der Schlüsselnamen ergeben sich aus den `input`-Tags.

Erläuterungen zum Beispiel 1/2

```
<form
    action="verarbeiteDaten.php"
    method="GET">
    <label for="vornameFeld">Vorname:</label><br>
    <input type="text" id="vornameFeld"
        name="vorname" value="Codie" required><br>
    <label for="nachnameFeld">Nachname:</label><br>
    <input type="text" id="nachnameFeld"
        name="nachname" value="Coder"><br><br>
    <input type="submit" value="Senden">
</form>
```

- Ein Formular wird mit dem "form"-Tag definiert.
- "action" legt die Seite fest, die beim Klick auf einen Submit-Button aufgerufen wird.
- method="GET" legt fest, dass die Formulardaten als Querystring an den Server gesendet werden. "POST" würde die Daten im Body des HTTP-Pakets senden.
- Der Text vor einem Eingabefeld wird mit "label" festgelegt. Mit dem "for"-Attribut wird das Label dem entsprechenden Eingabefeld zugeordnet. Ein Klick auf das Label setzt den Cursor in das Textfeld.

Erläuterungen zum Beispiel 2/2

```
<form action="verarbeiteDaten.php" method="get">
    <label for="vornameFeld">Vorname:</label><br>
    <input type="text" id="vornameFeld"
        name="vorname" value="Codie" required><br>
    <label for="nachnameFeld">Nachname:</label><br>
    <input type="text" id="nachnameFeld"
        name="nachname" value="Coder"><br><br>
    <input type="submit" value="Senden">
</form>
```

- 'input type="text"' definiert ein einzeiliges Texteingabefeld.
- Das Eingabefeld besitzt die Id "vornameFeld", welche bei der Label-Definition referenziert wird.
- Der durch "name" festgelegte Feldname wird bei der Datenübertragung als Schlüssel genutzt.
- "value" kann genutzt werden, um das Feld vorzubelegen.
- "required" bewirkt, dass das Formular erst gesendet wird, wenn das Feld ausgefüllt wurde. Wenn nicht, erscheint eine Fehlermeldung.
- 'input type="submit"' definiert einen Submit-Button.

Das Beispiel zeigt, dass ein Formular ein Container für Formularelemente ist. Es gibt viele Typen von Formularelementen, folgend eine Übersicht über die behandelten Formularelemente:

Formularelement	**HTML-Element**
Einzeiliges Textfeld	input type="text"
Mehrzeiliges Textfeld	textarea
Checkbox für Mehrfachauswahl	input type="checkbox"
Radiobuttons für "eins-aus-N"-Auswahl	input type="radio"
Auswahlliste (Combobox/Dropdown) für "eins-aus-N"-Auswahl	select
Buttons	input type="button"
weitere	input type= "number", "url", "email", "date", "color"

Anmerkung: eine Übersicht über HTML-Inputs findet man z.B. hier.



Best Practice für Formulare

- Form-Tag mit `action` und `method` Attribut, wenn diese leer sind dann nutzt der Browser als Defaultwert die aktuelle Seite als Aktionsziel und GET als Methode
- Alle Eingabemöglichkeiten (input, textarea, select) innerhalb des Form-Tags
- Zum Absenden des Formulars ein Button-Tag oder ein Input-Tag mit Type `submit`

TEXTFELDER

Es gibt einzeilige und mehrzeilige Textfelder. Beide Arten werden am Beispiel einer Einkaufsliste gezeigt.

```
<form action='verarbeiteDaten.php' method='get'>
  <label for='eintragFeld'>Neuer Eintrag:</label><br>
  <input type='text' id='eintragFeld' name='eintrag' ><br>
  <label>letzter Eintrag:</label><br>
  <input type='text' value='3L Milch' disabled><br><br>
  <input type='submit' value='Eintragen'>
</form>
<h3>Einkaufsliste</h3>
<textarea rows='2' cols='20'>
3L Milch
Butter
Brot
</textarea>
```

Erläuterung zur Einkaufsliste

- Die aktuelle Einkaufsliste wird in einem mehrzeiligen Textfeld angezeigt.
- Der jüngste Eintrag wird in einem nicht änderbaren Textfeld angezeigt.
- Durch Ausfüllen des ersten Textfeldes und Klick auf "Eintragen" wird der Text in die erste Zeile der Einkaufsliste eingefügt.

Hier der HTML-Code für das Beispiel zum Anlegen einer Einkaufsliste:

```
<form action="verarbeiteDaten.php" method="get">
  <label for="eintragFeld">Neuer Eintrag:</label><br>
  <input type="text" id="eintragFeld" name="eintrag" ><br>
  <label>letzter Eintrag:</label><br>
  <input type="text" value="3L Milch" disabled><br><br>
  <input type="submit" value="Eintragen">
</form>
<h3>Einkaufsliste</h3>
<textarea rows="2" cols="20">
3L Milch
Butter
Brot
</textarea>
```

- Das Feld für den letzten Eintrag kann nicht geändert werden. Anstelle von `disabled` kann auch `readonly` stehen. `readonly` erhält im Gegensatz zu `disabled` die Tabulator-Reihenfolge.
- Element `textarea` definiert das mehrzeilige Textfeld.
- Attribut `rows` und `cols` legen die sichtbare Größe des Feldes fest.
- Anmerkung: die Anwendung funktioniert mit HTML alleine nicht. Hierfür benötigen wir JavaScript.

CHECKBOXEN

Checkboxen dienen dazu, dem Benutzer die Auwahl einer oder mehrerer Optionen zu ermöglichen, wobei die Menge der Optionen klein sein sollte. Beispiel:

```
<fieldset>
    <legend>Welche Milchprodukte magst du?</legend>
    <input type='checkbox'
        id='butter'
        name='butter' value='magIch'>
    <label for='butter'>Butter</label><br>
    <input type='checkbox' id='milch' name='milch' value='magIch'>
    <label for='milch'>Milch</label><br>
    <input type='checkbox' id='sahne' name='sahne' value='magIch' checked>
    <label for='sahne'>Sahne</label><br>
</fieldset>
```

- Benutzer kann 1-3 Milchprodukte auswählen bzw. wieder abwählen.
- Es ist egal, ob auf die Checkbox oder das Label dahinter geklickt wird.

Folgender HTML-Code erzeugt das Beispiel auf der letzten Seite:

```
<fieldset>
    <legend>Welche Milchprodukte magst du?</legend>
    <input type="checkbox"
        id="butter"
        name="butter" value="magIch">
    <label for="butter">Butter</label><br>
    <input type="checkbox" id="milch" name="milch" value="magIch">
    <label for="milch">Milch</label><br>
    <input type="checkbox" id="sahne" name="sahne" value="magIch" checked>
    <label for="sahne">Sahne</label><br>
</fieldset>
```

- Element `fieldset` umrahmt die Checkbox-Gruppe, `legend` definiert den Titel.
- Attribut `type="checkbox"` definiert eine Checkbox.
- Das id-Attribut wird zur Referenz durch das Label (s. nächstes Tag) benötigt.
- Attribut `name` bzw. `value` definieren das beim "Submit" zu sendende Schlüssel/Wert-Paar, falls die Checkbox ausgewählt wurde.
- Attribut `checked` zeigt die Checkbox selektiert an.

RADIOBUTTONS

Radiobuttons dienen dazu, dem Benutzer die Auwahl einer Option aus einer kleinen Auswahlmenge zu ermöglichen. Beispiel:

```
<p>Wähle dein Alter:</p>
<input type='radio' id='age1'
    name='age'
    value='30'>
<label for='age1'>0 - 30</label><br>
<input type='radio' id='age2' name='age' value='60'>
<label for='age2'>31 - 60</label><br>
<input type='radio' id='age3' name='age' value='100'>
<label for='age3'>61 - 100</label><br><br>
```

- Der Benutzer kann sein/ihr Alter auswählen bzw. wieder abwählen (mit STRG-Taste).
- Es ist dabei egal, ob auf den Radiobutton oder das Label dahinter geklickt wird.

Folgender HTML-Code erzeugt das Beispiel auf der letzten Seite:

```
<p>Wähle dein Alter:</p>
<input type="radio" id="age1"
    name="age"
    value="30">
<label for="age1">0 - 30</label><br>
<input type="radio" id="age2" name="age" value="60">
<label for="age2">31 - 60</label><br>
<input type="radio" id="age3" name="age" value="100">
<label for="age3">61 - 100</label><br><br>
```

- Attribut `type="radio"` definiert einen RadioButton (hier mit id "age1").
- Attribut `name` legt den gemeinsamen Namen aller zusammengehörigen Radiobuttons fest.
- Attribut `value` definiert den beim "Submit" mit dem Namen zu sendenden Wert. Gesendet wird der Wert, dessen Radiobutton ausgewählt wurde.

COMBOBOXEN

Comboboxen werden genutzt, um ein Element aus einer Liste auszuwählen. Die Liste wird platzsparend als Klappliste dargestellt und nur für den Auswahlvorgang selbst aufgeklappt. Beispiel:

```
<label for='cars'>Wähle einen Autohersteller:</label>
<select id='cars'
    name='cars'>
    <option value='audi'>Audi</option>
    <option value='bmw'>BMW</option>
    <option value='mercedes' selected>Mercedes</option>
    <option value='opel'>Opel</option>
</select>
```

- Der Benutzer kann mit der Maus eine Auswahl treffen.
- Die Listenelemente können auch gruppiert werden, siehe z.B. W3Schools.

Folgender HTML-Code erzeugt das Beispiel auf der letzten Seite:

```
<label for="cars">Wähle einen Autohersteller:</label>
<select id="cars"
    name="cars">
    <option value="audi">Audi</option>
    <option value="bmw">BMW</option>
    <option value="mercedes" selected>Mercedes</option>
    <option value="opel">Opel</option>
</select>
```

- Element `select` definiert eine Combobox (hier mit id "cars").
- Der gewählte Eintrag wird bei Formular-Submit als `<name> = <value>` an den Server gesendet.
- Angezeigt wird "Audi", beim "Submit" wird `cars=audi` übertragen.
- Attribut `selected` legt fest, welcher Eintrag zu Beginn ausgewählt sein soll (Default: der erste).

BUTTONS

- Zur Nutzung von Buttons gibt es das `input`- und das `button`-Tag. Der `button`-Tag ist zu bevorzugen, da der Buttoninhalt damit flexibler gestaltet werden kann (z.B. durch eine eingebettete Grafik).
- Das Styling von Buttons wird im Kapitel zu CSS erklärt.

Erläuterung einiger Button-Attribute am Beispiel (1/2):

```
<input type="button" value="login">
<button type="submit" formaction="pruefeEingaben.php">login</button>
<input type="submit" formnovalidate value="abbrechen">
```

- Attribut `type` kann folgende Werte annehmen: `button` (Ereignisbehandlung mit JavaScript), `submit` (löst Formular-Submit aus), `reset` (setzt Formulareingaben zurück)
- Für submit-buttons kann eine individuelle `action` gesetzt werden. Dies ist vorteilhaft, wenn man mehrere Buttons in einem Formular definiert.

Erläuterung einiger Button-Attribute am Beispiel (2/2):

```
<input type="button" value="login">
<button type="submit" formaction="pruefeEingaben.php">login</button>
<input type="submit" formnovalidate value="abbrechen">
```

- Beachte: Default-`type` bei `button`-Tags ist `submit`!
- Die Validierung von Eingabefeldern (z.B. required-Attribut für Muss-Felder) wird durch `formnovalidate` bei Klick auf den Button nicht durchgeführt. Dies ist z.B. bei Abbrechen-Buttons vorteilhaft.

ÜBUNG: GRUNDLEGENDE FORMULAR-TAGS

Welche Gruppe enthält **nur** Elemente, die typischerweise in HTML-Formularen verwendet werden?

- [ ] <form>, <input>, <label>, ,
- [ ] <form>, <field>, <input>, <option>, <submit>
- [ ] <form>, <input>, <label>, <select>, <textarea>
- [ ] <input>, <select>, <checkbox>, <radio>, <button>

ÜBUNG: POST VS. GET

Die POST-Methode in einem Formular ist sicherer für die Übertragung sensibler Daten als die GET-Methode, da die Daten nicht in der URL sichtbar sind.

- [ ] Richtig
- [ ] Falsch

ÜBUNG: WECHSELWIRKUNG FORM, INPUT UND HTTP

Erklären Sie die Wechselwirkung zwischen der `method`-Eigenschaft und `action`-Eigenschaft eines HTML-Formulars, den `name`-Attributen der Input-Elemente und den resultierenden HTTP-Requests.

Geben Sie eine eigene Antwort, eine Beispielantwort wird nach dem Absenden angezeigt



3.9 ÜBUNG

